

Efficient Model Transformation of Control Flow Graphs into BPMN Diagrams

Alvaro Gallardo

Damian Arellanes

Abstract

This report studies the transformation of control flow graphs into BPMN diagrams by means of Double-Pushout (DPO) graph rewriting. The starting point is the *computon* model introduced in [1], where control flow and data flow are explicitly separated in a compositional setting. In particular, the control flow graphs considered in this report are not arbitrary graphs: they are extracted from computons through the control-flow functor $\mathcal{C}$ provided in [1].

The problem addressed here is that, although computons provide a compositional representation of high-level computations with explicit control flow, many software engineering tools and workflow platforms rely on BPMN as the standard notation for visualising and analysing process structure. Hence, a correct and systematic transformation from computon-derived control flow graphs into BPMN diagrams is required. This is important because it enables the analysis of execution order in a widely adopted notation while preserving the semantics of the original control structure.

To address this problem, we define a collection of graph rewriting rules and study their application using the DPO approach. The rules are designed to transform typed control flow graph patterns into their BPMN counterparts, including tasks, gateways, events and sequence flows. We also discuss termination arguments for the rules considered in this report and outline how the rules have been implemented in the Groove tool for experimentation.

The overall goal is not merely to rewrite arbitrary graphs, but to support the transformation of hierarchically structured control flow representations arising from computons. For this reason, the report should be understood in the broader setting of nested graph rewriting rather than plain graph rewriting.

1 Introduction

Business Process Model and Notation (BPMN) is a standard language for representing workflow control structures in a form that is both visual and widely used in software engineering practice. At the same time, recent work on models of high-level computation has proposed the *computon model* as a compositional formalism in which control flow and data flow are treated as distinct but related dimensions [1]. In that model, a computation is represented by a computon, and its control flow graph is extracted through the functor $\mathcal{C}$ defined in [1]. This report takes that formal setting as its starting point.

The concrete problem addressed here is the following: given a control flow graph extracted from a computon, how can one transform it into an equivalent BPMN diagram in a correct and systematic way? This problem is relevant for at least three reasons. First, BPMN remains one of the most widely used notations for describing process structure in both academia and industry [5]. Second, computons provide a compositional and semantically explicit account of control flow, but this representation is not directly the one expected by BPMN-based tooling. Third, once a transformation is available, it becomes possible to analyse compositional control structures using a standard process notation without having to reconstruct control information from mixed control/data models.

The proposed solution is to model the transformation as a graph rewriting problem. More precisely, we work in a typed graph setting and define rewriting rules using the Double-Pushout

(DPO) approach. These rules are intended to transform specific control flow graph patterns into BPMN constructs such as tasks, XOR gateways, AND gateways, start events, end events and sequence flows. Since the control flow graphs considered in this report are extracted from computons, the transformation is grounded in the explicit separation of control flow and data flow established in [1]. In that sense, the report should not be read as a generic graph-to-graph conversion exercise, but as a semantics-aware transformation from computon-derived control structures into BPMN.

A further aspect of the problem is that computons are compositional and may give rise to hierarchical or boxed structures. Consequently, the transformation problem is more naturally framed in terms of nested graph rewriting rather than plain graph rewriting. Although the present report focuses on the rewriting rules themselves, this broader perspective is important for understanding the intended scope of the project.

This report is structured as follows. Section 2 introduces the graph-theoretic and DPO-rewriting preliminaries used throughout the text, together with formal definitions of typed control flow graphs and BPMN diagrams. Sections 3–6 present the rewriting rules considered in this report, first in graphical form and then in formal DPO form, together with termination arguments where appropriate. Section 7 discusses applicability conditions and structural invariants of the rules. Section 8 briefly describes the current implementation in Groove and provides the project repository. Finally, Section 9 summarises the main contributions of the report and outlines directions for future work.

2 Preliminaries: Double-Pushout (DPO) Rewriting

We work with finite directed typed graphs. Since the control flow graphs studied in this report are extracted from computons through the functor $\mathcal{C}$ of [1], they are not arbitrary graphs but typed control-flow representations of computon structure. We adopt the Double-Pushout (DPO) approach to graph rewriting, a standard algebraic framework for the specification of graph transformations [2, 3].

Let Σ_V be a finite set of vertex labels and Σ_E a finite set of edge labels.

Definition 2.1 (Typed Graph). *A typed graph G is a tuple*

$$G = (V_G, E_G, \text{src}_G, \text{tgt}_G, \text{lab}_G^V, \text{lab}_G^E),$$

where V_G is a finite set of vertices, E_G is a finite set of edges, $\text{src}_G, \text{tgt}_G : E_G \rightarrow V_G$ are the source and target functions, $\text{lab}_G^V : V_G \rightarrow \Sigma_V$ is the vertex-labelling function, and $\text{lab}_G^E : E_G \rightarrow \Sigma_E$ is the edge-labelling function.

Definition 2.2 (Graph Morphism). *Let*

$$G = (V_G, E_G, \text{src}_G, \text{tgt}_G, \text{lab}_G^V, \text{lab}_G^E) \quad \text{and} \quad H = (V_H, E_H, \text{src}_H, \text{tgt}_H, \text{lab}_H^V, \text{lab}_H^E)$$

be typed graphs. A graph morphism $f : G \rightarrow H$ is a pair of functions

$$f_V : V_G \rightarrow V_H, \quad f_E : E_G \rightarrow E_H$$

such that, for every edge $e \in E_G$,

$$\text{src}_H(f_E(e)) = f_V(\text{src}_G(e)), \quad \text{tgt}_H(f_E(e)) = f_V(\text{tgt}_G(e)),$$

and labels are preserved, that is,

$$\text{lab}_H^V(f_V(v)) = \text{lab}_G^V(v) \quad \text{for all } v \in V_G,$$

$$\text{lab}_H^E(f_E(e)) = \text{lab}_G^E(e) \quad \text{for all } e \in E_G.$$

Definition 2.3 (BPMN Diagram). *A BPMN diagram is a typed multipartite directed graph*

$$B = (E_B, T_B, G_B, F_B, \text{src}_B, \text{tgt}_B, \tau_B),$$

where:

- (i) E_B, T_B and G_B are finite pairwise disjoint sets of nodes, respectively called events, tasks and gateways;
- (ii) F_B is a finite set of directed edges called sequence flows;
- (iii) $\text{src}_B, \text{tgt}_B : F_B \rightarrow E_B \cup T_B \cup G_B$ are the source and target functions;
- (iv) $\tau_B : E_B \cup T_B \cup G_B \rightarrow \Sigma_{\text{BPMN}}$ is a typing function such that

$$\tau_B(E_B) \subseteq \{\text{StartEvent}, \text{EndEvent}\},$$

$$\tau_B(T_B) \subseteq \{\text{Task}\},$$

$$\tau_B(G_B) \subseteq \{\text{XOR}, \text{AND}\}.$$

Definition 2.4 (DPO Rewriting Rule). *A DPO rewriting rule is a span of graph morphisms*

$$p = (L \xleftarrow{l} K \xrightarrow{r} R),$$

where L is the left-hand side, R is the right-hand side, and K is the interface graph. The morphism $l : K \rightarrow L$ specifies the part of L that is preserved, and the morphism $r : K \rightarrow R$ specifies how the preserved part is embedded into R .

Definition 2.5 (Match). *Let*

$$p = (L \xleftarrow{l} K \xrightarrow{r} R)$$

be a DPO rewriting rule and let G be a host graph. A match of p in G is a graph morphism $m : L \rightarrow G$ such that the pushout complement of l along m exists.

Definition 2.6 (DPO Step). *Let*

$$p = (L \xleftarrow{l} K \xrightarrow{r} R)$$

be a DPO rewriting rule, let G be a host graph, and let $m : L \rightarrow G$ be a match. A DPO step

$$G \Rightarrow_{p,m} H$$

is obtained by first computing the pushout complement of l along m , and then computing the pushout of the resulting morphism with r .

Let $\mathbf{Graph}_{\Sigma}^{\text{fin}}$ denote the class of all finite typed graphs over the fixed alphabets under consideration.

Definition 2.7 (Termination). *A rewriting rule p is terminating if there is no infinite sequence*

$$G_0 \Rightarrow_p G_1 \Rightarrow_p G_2 \Rightarrow_p \dots$$

with $G_i \in \mathbf{Graph}_{\Sigma}^{\text{fin}}$ for all i . Equivalently, p is terminating if there exists a well-founded measure

$$\mu : \mathbf{Graph}_{\Sigma}^{\text{fin}} \rightarrow \mathbb{N}$$

such that $\mu(G) > \mu(H)$ for every rewriting step $G \Rightarrow_p H$.

Typing convention for control flow graphs. In the sequel, we use a finite set of node types

$$\Sigma_V^{\text{CFG}} = \{0, \kappa, \text{Task}, \text{XOR}, \text{AND}\}.$$

Here, 0 denotes control ports, κ denotes computation units in the control flow graph extracted from a computon, and Task, XOR, AND denote BPMN-level node types introduced by rewriting. Thus, expressions such as $u : 0$ or $v : \kappa$ indicate that u belongs to the set of 0-nodes and v belongs to the set of κ -nodes.

3 Rewriting Rules

In this section, each rewriting rule is presented first in graphical form and then in formal DPO form. This ordering is intentional: the diagrams make the intended transformation immediately clear, while the formal definitions specify the exact graphs and morphisms involved. The rules shown in the figures have been implemented and tested in the Groove graph transformation tool [4]. The graphical rule depictions included below correspond to that implementation.

4 Rule 1: Intermediate 0-Node Elimination

This rule removes an intermediate 0-node occurring between two κ -nodes and replaces the corresponding two-edge fragment by a direct connection.

4.1 Graphical presentation

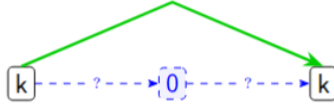


Figure 1: Graphical presentation of Rule 1.

4.2 Formal definition

Definition 4.1 (Rule 1). *Rule 1 is the DPO rule*

$$p_1 = (L_1 \xleftarrow{l_1} K_1 \xrightarrow{r_1} R_1),$$

defined as follows.

The graph L_1 has vertices

$$\{u, k_1, v, k_2, w\},$$

with typing

$$u : 0, \quad k_1 : \kappa, \quad v : 0, \quad k_2 : \kappa, \quad w : 0,$$

and edges

$$u \rightarrow k_1, \quad k_1 \rightarrow v, \quad v \rightarrow k_2, \quad k_2 \rightarrow w.$$

The interface graph K_1 has vertices

$$\{u, k_1, k_2, w\},$$

with the same types as above, and edges

$$u \rightarrow k_1, \quad k_2 \rightarrow w.$$

The graph R_1 has vertices

$$\{u, k_1, k_2, w\},$$

with typing

$$u : 0, \quad k_1 : \kappa, \quad k_2 : \kappa, \quad w : 0,$$

and edges

$$u \rightarrow k_1, \quad k_1 \rightarrow k_2, \quad k_2 \rightarrow w.$$

The morphisms $l_1 : K_1 \rightarrow L_1$ and $r_1 : K_1 \rightarrow R_1$ are the obvious inclusions on preserved vertices and edges.

4.3 Termination

Proposition 4.1. *Rule 1 is terminating on finite typed graphs.*

Proof. Let $\nu(G)$ be the number of vertices of G labelled 0 that occur as intermediate 0-nodes in matches of Rule 1. Since G is finite, $\nu(G) \in \mathbb{N}$. Each application of Rule 1 removes exactly one such intermediate 0-node and does not create any new one of the same form. Hence, if $G \Rightarrow_{p_1} H$, then $\nu(H) < \nu(G)$. Therefore, no infinite rewriting sequence can exist. $\square$

5 Rule 2: Branch Structuring into XOR

This rule replaces an unstructured branch from a 0-node to several κ -nodes by an explicit XOR-based BPMN fragment.

5.1 Graphical presentation

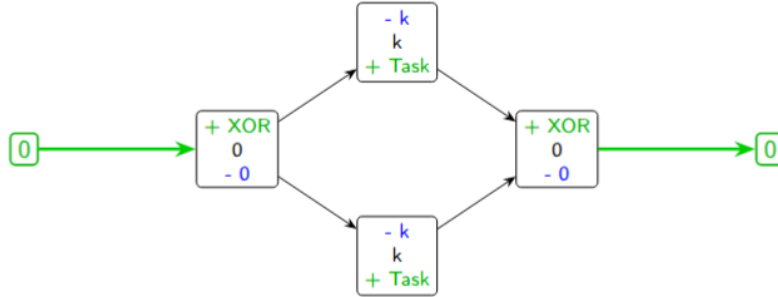


Figure 2: Graphical presentation of Rule 2.

5.2 Formal definition

Definition 5.1 (Rule 2). *For simplicity, we present the binary case. The n -ary case is analogous.*

Rule 2 is the DPO rule

$$p_2 = (L_2 \xleftarrow{l_2} K_2 \xrightarrow{r_2} R_2).$$

The graph L_2 has vertices

$$\{z, a, b\},$$

with types

$$z : 0, \quad a : \kappa, \quad b : \kappa,$$

and edges

$$z \rightarrow a, \quad z \rightarrow b.$$

The interface graph K_2 has the single preserved vertex

$$\{z\},$$

with type $z : 0$.

The graph R_2 has vertices

$$\{z, x_s, t_1, t_2, x_j, z'\},$$

with types

$$z : 0, \quad x_s : \text{XOR}, \quad t_1 : \text{Task}, \quad t_2 : \text{Task}, \quad x_j : \text{XOR}, \quad z' : 0,$$

and edges

$$z \rightarrow x_s, \quad x_s \rightarrow t_1, \quad x_s \rightarrow t_2, \quad t_1 \rightarrow x_j, \quad t_2 \rightarrow x_j, \quad x_j \rightarrow z'.$$

The morphisms l_2 and r_2 identify the preserved 0-node z .

5.3 Termination

Proposition 5.1. *Rule 2 is terminating on finite typed graphs.*

Proof. Rule 2 does not preserve a left-hand-side occurrence of the form $0 \rightarrow \kappa$. Indeed, every application removes such an occurrence and replaces it with a fragment whose internal nodes are of types **XOR** and **Task**, not κ . Therefore, once a specific unstructured branch has been rewritten, the same match cannot reappear. Since the host graph is finite and the number of left-hand-side occurrences of the form captured by Rule 2 is finite, Rule 2 can only be applied a finite number of times. Hence, it is terminating. $\square$

6 Rule 3: $\kappa\kappa$ -Diamond to AND/Task/AND with 0-Node Additions

This rule transforms a diamond-shaped control-flow pattern into an explicit BPMN-style structure. The first and last κ -nodes are relabelled as **AND**, the upper κ -node is relabelled as **Task**, the lower κ -node remains unchanged, and two 0-nodes are added at the boundary.

6.1 Graphical presentation

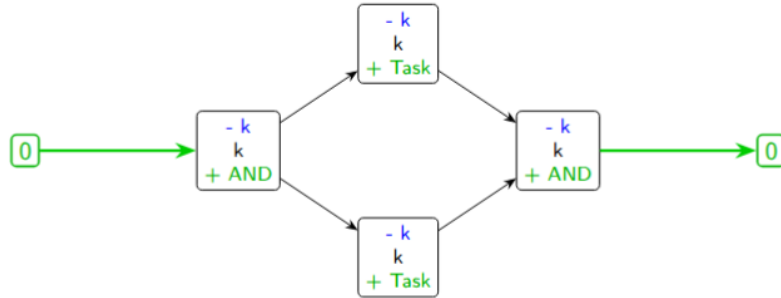


Figure 3: Graphical presentation of Rule 3.

6.2 Formal definition

Definition 6.1 (Rule 3). *Rule 3 is the DPO rule*

$$p_3 = (L_3 \xleftarrow{l_3} K_3 \xrightarrow{r_3} R_3).$$

The graph L_3 has vertices

$$\{a, b, c, d\},$$

all typed κ , with edges

$$a \rightarrow b, \quad a \rightarrow c, \quad b \rightarrow d, \quad c \rightarrow d.$$

The interface graph K_3 preserves the whole diamond:

$$V_{K_3} = \{a, b, c, d\}, \quad E_{K_3} = \{a \rightarrow b, a \rightarrow c, b \rightarrow d, c \rightarrow d\}.$$

The graph R_3 has vertices

$$\{a, b, c, d, z_a, z_d\},$$

with typing

$$a : \text{AND}, \quad b : \text{Task}, \quad c : \kappa, \quad d : \text{AND}, \quad z_a : 0, \quad z_d : 0,$$

and edges

$$a \rightarrow b, \quad a \rightarrow c, \quad b \rightarrow d, \quad c \rightarrow d, \quad z_a \rightarrow a, \quad d \rightarrow z_d.$$

The morphism $l_3 : K_3 \rightarrow L_3$ is the inclusion, and the morphism $r_3 : K_3 \rightarrow R_3$ preserves the original diamond while relabelling a, b, d as above.

6.3 Termination of Rule 3

Proposition 6.1. *Rule 3 is terminating on finite typed graphs.*

Proof. Rule 3 is applicable only to occurrences of a diamond-shaped pattern whose four internal nodes are typed κ . After one application, the first and last κ -nodes are relabelled as **AND**, the upper κ -node is relabelled as **Task**, and two 0-nodes are added. Consequently, the rewritten subgraph no longer matches the left-hand side of Rule 3. Since a finite graph contains only finitely many such $\kappa\kappa$ -diamond occurrences, Rule 3 can be applied only finitely many times. Therefore, Rule 3 is terminating. $\square$

7 Rule 4: Relabelling the Remaining κ -Node as Task

This rule takes the result of Rule 3 and performs a final relabelling step by changing the remaining lower κ -node into a **Task**-node. The rule is purely relabelling: no nodes or edges are added or deleted.

7.1 Graphical presentation

7.2 Formal definition

Definition 7.1 (Rule 4). *Rule 4 is the DPO rule*

$$p_4 = (L_4 \xleftarrow{l_4} K_4 \xrightarrow{r_4} R_4).$$

The graph L_4 has vertices

$$\{a, b, c, d, z_a, z_d\},$$

with typing

$$a : \text{AND}, \quad b : \text{Task}, \quad c : \kappa, \quad d : \text{AND}, \quad z_a : 0, \quad z_d : 0,$$

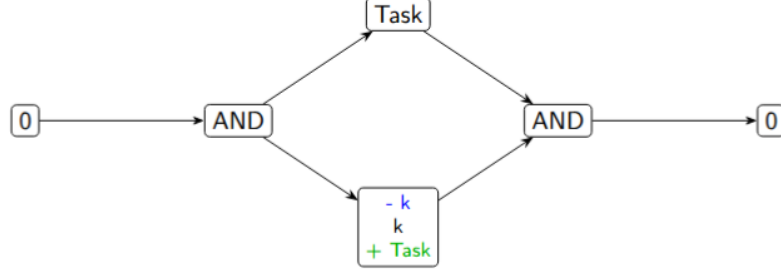


Figure 4: Graphical presentation of Rule 4.

and edges

$$z_a \rightarrow a, \quad a \rightarrow b, \quad a \rightarrow c, \quad b \rightarrow d, \quad c \rightarrow d, \quad d \rightarrow z_d.$$

The interface graph K_4 preserves the whole graph L_4 .

The graph R_4 has the same vertices and edges as L_4 , but with the relabelling

$$c : \kappa \mapsto c : \text{Task}.$$

The morphism $l_4 : K_4 \rightarrow L_4$ is the identity inclusion, and $r_4 : K_4 \rightarrow R_4$ preserves all structure while relabelling c .

7.3 Termination of Rule 4

Proposition 7.1. *Rule 4 is terminating on finite typed graphs.*

Proof. Rule 4 is a pure relabelling rule: it changes the label of the remaining lower κ -node to **Task** and does not add or delete vertices or edges. After one application, the resulting subgraph no longer contains the κ -label required by the left-hand side of Rule 4. Hence the same occurrence cannot be rewritten again. Since the host graph is finite, there are only finitely many such occurrences, and Rule 4 is terminating. $\square$

8 Remarks on Applicability and Invariants

The DPO construction for each rule requires the usual gluing conditions, including dangling and identification conditions [3, 2]. In the present setting, these conditions ensure that rewriting steps preserve graph well-formedness.

For Rule 1, deletion is confined to the intermediate 0-node captured by the left-hand side. For Rule 2, the deleted elements are the direct unstructured edges from a 0-node to κ -nodes. In both cases, applicability depends on whether the corresponding pushout complement exists. If necessary, one can refine applicability further by introducing negative application conditions.

Rules 3 and 4 are more conservative. Rule 3 preserves the original diamond structure while adding boundary 0-nodes and relabelling selected κ -nodes. Rule 4 is purely relabelling. Hence, these two rules preserve the underlying shape of the matched control-flow fragment while refining its typing towards BPMN constructs.

More generally, all four rules respect the intended metamodel. Newly introduced nodes are assigned types consistent with the target BPMN representation, and preserved nodes remain well-typed under the adopted control-flow graph alphabet. This is important because the transformation is not merely syntactic: it is meant to turn control flow graphs extracted from computons into BPMN diagrams in a type-consistent way.

9 Code Availability

The rewriting rules described in this report have been implemented in the Groove graph transformation tool [4]. In particular, the graphical rule representations shown in the previous sections correspond to Groove rules used for experimentation and validation.

The current implementation is available on GitHub at:

<https://github.com/alvarow90/groove-box-evaluator/tree/main>

The repository contains the current rule set and supporting material for evaluating rewriting instances. Since the broader objective of the project involves boxed or hierarchical structures, the implementation should be understood as an initial step towards nested graph rewriting support rather than as a final transformation engine.

10 Conclusion

This report has presented a DPO-based approach to transforming control flow graphs into BPMN diagrams. The theoretical point of departure is the computon model of [1], where control flow and data flow are explicitly separated and where control flow graphs are extracted by means of the functor $\mathcal{C}$. Framing the problem in this way is important because it gives the transformation a semantic basis: the source graphs are not arbitrary, but arise from a compositional model of high-level computation.

On that basis, we introduced typed graph-theoretic preliminaries, formalised BPMN diagrams in graph terms, and specified several rewriting rules in the DPO framework. The rules were presented both graphically and formally in order to make their intended effect and their algebraic content equally clear. Termination arguments were also provided for the rules considered in this report. In particular, some rules terminate by strict structural decrease, while others terminate because their application removes the very pattern required for reapplication.

From a practical perspective, the work reported here contributes towards a systematic bridge between computon-derived control flow representations and BPMN diagrams. Such a bridge is useful because it allows compositional control structures to be visualised and analysed in a standard notation already widely used in software engineering. Moreover, the use of Groove provides an initial executable setting in which the rules can be tested and refined.

Several directions remain open. First, the current report should be extended with a more general and fully formal treatment of hierarchical or boxed structures, since the intended application domain is closer to nested graph rewriting than to flat graph rewriting. Second, the present rule set should be expanded to cover a broader class of control flow graph patterns extracted from computons. Third, correctness results stronger than termination should be studied, including confluence, semantic preservation and behavioural correspondence between source control flow graphs and target BPMN diagrams. Finally, the relationship between the present transformation system and the graph transformation approach outlined in [1] deserves to be developed in more detail.

References

- [1] Damian Arellanes. Compositional separation of control flow and data flow. *Journal of Logical and Algebraic Methods in Programming*, 150:101125, 2026.
- [2] Reiko Heckel and Gabriele Taentzer. *Graph Transformation for Software Engineers: With Applications to Model-Based Development and Domain-Specific Language Engineering*. Springer, 2020.

- [3] Hartmut Ehrig, Karsten Ehrig, Ulrike Prange, and Gabriele Taentzer. *Fundamentals of Algebraic Graph Transformation*. Springer, 2006.
- [4] A. H. Ghamarian, M. de Mol, A. Rensink, E. Zambon, and M. Zimakova. Modelling and analysis using GROOVE. *International Journal on Software Tools for Technology Transfer*, 14(1):15–40, 2012.
- [5] Object Management Group. *Business Process Model and Notation (BPMN), Version 2.0*. 2011.
- [6] Annegret Habel and Detlef Plump. Relabelling in graph transformation. In *Graph Transformation*, pages 135–147. Springer, 2002.